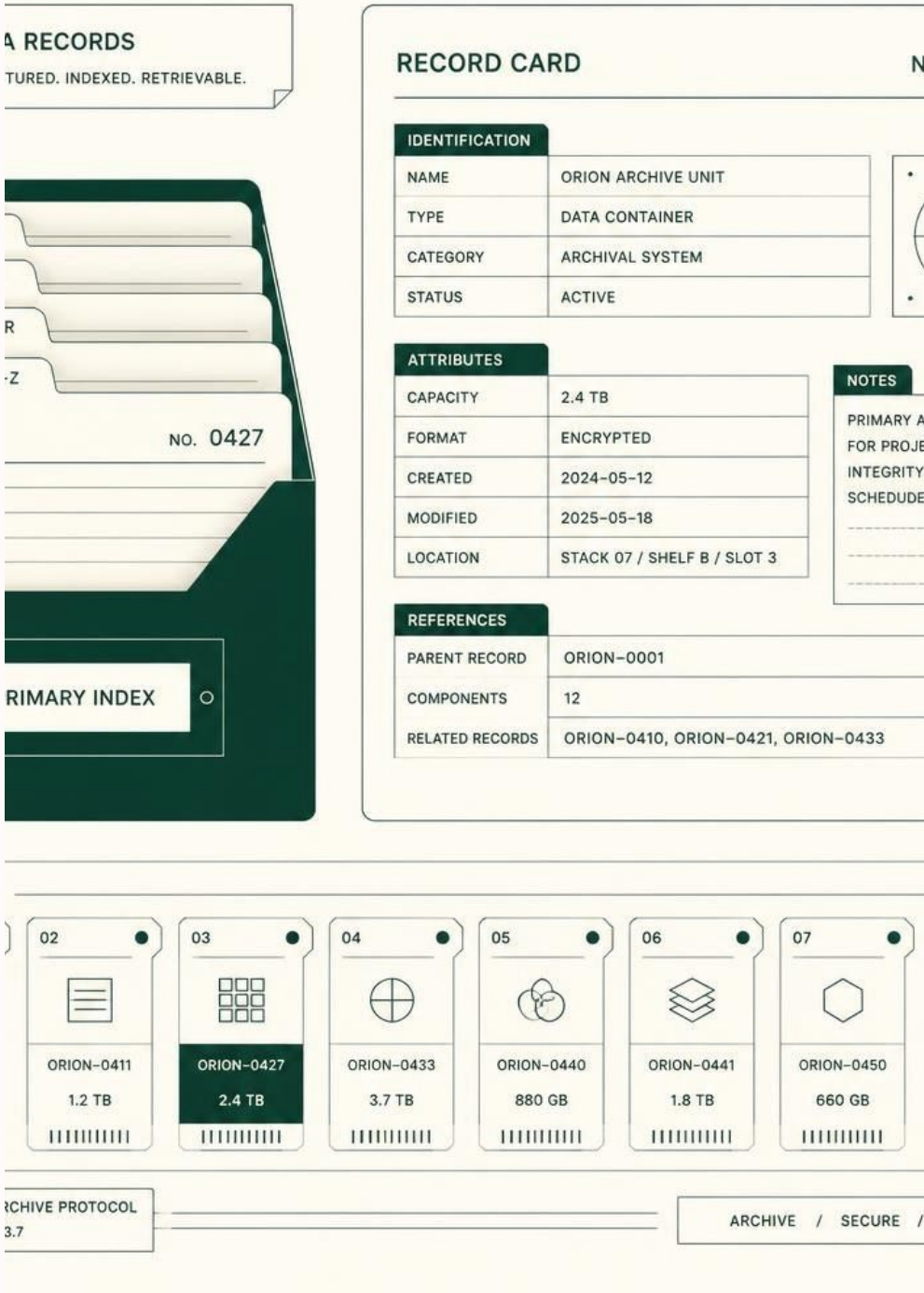
Struct in C

Dichiarazione e accesso, assegnamento e confronto, array di struct

LINGUAGGIO C | STRUTTURE E UNIONI | STRUCT

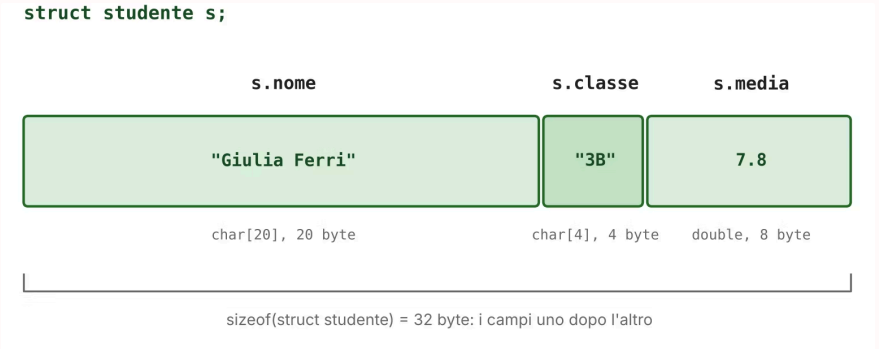


Un Nuovo Tipo Fatto di Campi

Una struct raggruppa in una sola variabile dati di tipo diverso che descrivono la stessa cosa. Ogni dato è un campo, e si raggiunge con l'operatore punto.

```
struct studente {
    char nome[20];
    char classe[4];
    double media;
};                          /* ; obbligatorio */

struct studente s;
strcpy(s.nome, "Giulia Ferri");
s.media = 7.8;
```



Un campo si usa come una variabile del suo tipo: s.nome è un array di char (si riempie con strcpy), s.media è un double.

Inizializzare una Struct

```
struct tessera sara = {1024, "Bianchi Sara", 12, 25.50};
struct tessera nuova = {.credito = 10.00, .numero = 1025};
struct tessera vuota = {0};

vuota = {2000, "Neri Luca", 0, 5.0};    /* errore */
vuota.numero = 2000;                    /* corretto */
```

Forma	Esempio	Campi non indicati
in ordine	{1024, "Bianchi Sara", 12, 25.50}	a zero
con il nome	{.numero = 1025, .credito = 10.00}	a zero
tutto a zero	{0}	—

> La lista tra graffe vale solo nella dichiarazione. Una struct locale non inizializzata contiene valori casuali.

typedef e struct: un Tipo Davvero Nuovo

```
typedef struct {
    double valore;
} Euro;

typedef struct {
    double valore;
} Chilometri;

Euro prezzo = {1.79};
Chilometri distanza = {290};
prezzo = distanza;
/* error: incompatible types when assigning
   to type 'Euro' from type 'Chilometri' */
```

Nel sottocapitolo typedef, Euro e Chilometri erano due nomi dello stesso double. Ogni struct invece è un tipo diverso da tutti gli altri, anche con gli stessi campi.

> Da qui in poi il corso usa typedef struct { ... } Nome; con l'iniziale maiuscola. La forma struct studente resta valida.

Copiare una Struct con =

Due struct dello stesso tipo si assegnano con =: tutti i campi vengono copiati, compresi gli array contenuti nella struct. Dopo la copia le due variabili sono indipendenti.

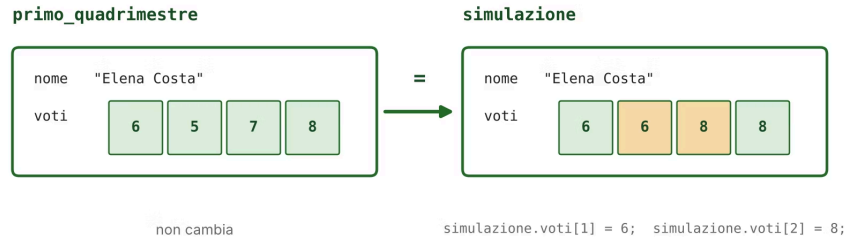
```
Pagella simulazione;

simulazione = primo_quadrimestre;
simulazione.voti[1] = 6;      /* l'originale non cambia */

int voti[4];
int copia_voti[4];
copia_voti = voti;            /* errore: un array non si assegna */
```

> Scambiare due struct intere scambia tutti i campi: per spostare due studenti si scambiano solo fila e pc, non i nomi.

```
simulazione = primo_quadrimestre;   /* copia di tutti i campi, array compreso */
```



Dopo la copia le due struct sono indipendenti.

Con due array: `w = v;` non compila.

Confrontare Campo per Campo

Il problema: a == b tra due struct non compila.

```
/* if (inventario == conteggio)    errore */

stesso_articolo = strcmp(inventario.codice, conteggio.codice) ==
0;
identici = stesso_articolo &&
        strcmp(inventario.descrizione, conteggio.descrizione) ==
0 &&
        inventario.quantita == conteggio.quantita;
```

La soluzione: si confrontano i campi, ognuno nel modo adatto al suo tipo: i numeri con ==, le stringhe con strcmp.

Operazione	Array	Struct
copia	ciclo o strcpy	b = a
confronto	ciclo o strcmp	campo per campo
scambio	ciclo con appoggio	temp = a; a = b; b = temp;

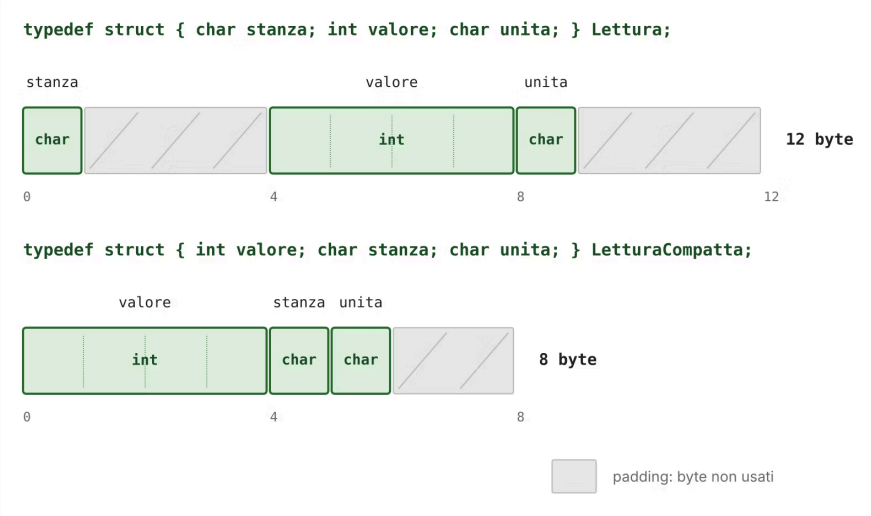
a.codice == b.codice confronta gli indirizzi di due array diversi: è sempre falso anche quando il testo è uguale.

Dimensione e Allineamento

Il compilatore può lasciare byte vuoti (padding) tra i campi, così che un int stia a un indirizzo multiplo di 4. La sizeof di una struct può essere maggiore della somma dei campi.

```
typedef struct {
    char stanza;
    int valore;
    char unita;
} Lettura;          /* 12 byte */

offsetof(Lettura, valore)    /* 4, da <stddef.h> */
```



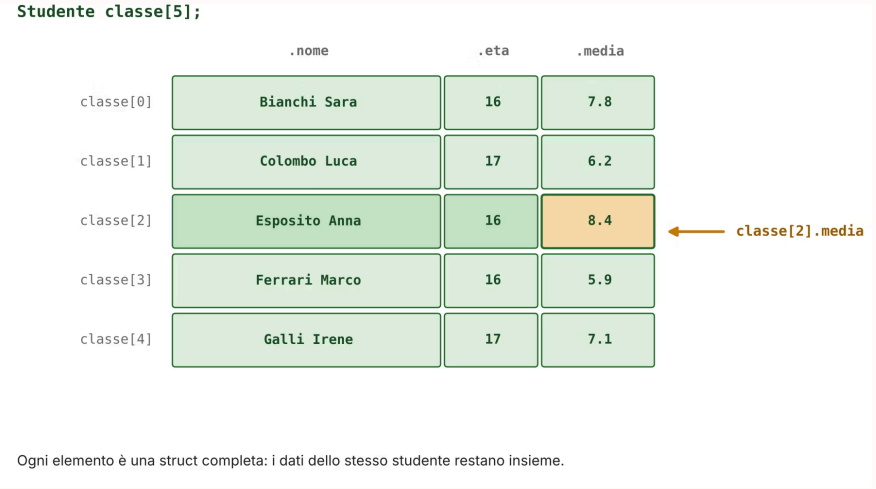
> Mettendo i campi grandi per primi il padding si riduce: per 10000 letture 80000 byte invece di 120000.

Array di Struct

Una struct descrive un oggetto, un array di struct un elenco: le righe di una tabella. Ogni elemento è una struct completa.

```
Studente classe[5] = {
    {"Bianchi Sara", 16, 7.8},
    {"Colombo Luca", 17, 6.2},
    ...
};

classe[i]          /* lo studente i */
classe[i].media    /* un suo campo */
classe[i].nome[0]  /* l'iniziale del nome */
```



> Al posto di tre array paralleli da tenere allineati c'è un solo array: i dati dello stesso studente restano insieme.

Ricerca e Massimo su un Campo

Gli algoritmi visti con gli array si applicano scegliendo il campo su cui lavorare. Il risultato di una ricerca è l'indice: con l'indice si leggono tutti gli altri campi.

```
i = 0;
while (i < LIBRI && trovato == -1) {
    if (strcmp(libri[i].titolo, cercato) == 0) {
        trovato = i;
    }
    i++;
}

if (libri[i].prestiti > libri[piu_letto].prestiti) {
    piu_letto = i;
}
```

Operazione	Condizione sul campo
ricerca	strcmp(libri[i].titolo, cercato) == 0
massimo	libri[i].prestiti > libri[migliore].prestiti
conteggio	libri[i].copie > libri[i].in_prestito

Struct che Contengono Array

```
typedef struct {
    char nome[20];
    int voti[MAX_VOTI];
    int n_voti;
} Registro;

Registro classe[STUDENTI];

classe[s].voti[v]         /* voto v dello studente s */
classe[s].voti[classe[s].n_voti] = 7;
classe[s].n_voti++;
```

Un campo può essere un array: per ogni studente i voti delle verifiche e quanti voti ha. Si legge da sinistra a destra: elemento dell'array esterno, campo, elemento del campo.

> Prima di aggiungere un voto si controlla che n_voti sia minore di MAX_VOTI. La stessa idea vale per le prenotazioni delle aule: char prenotata[ORE][4] dentro ogni Aula.

Errori Comuni da Evitare

1

Dimenticare il punto e virgola dopo la dichiarazione della struct.

2

Usare il nome del tipo al posto della variabile: Studente.media invece di s.media.

3

Assegnare una stringa a un campo array con = invece di strcpy.

4

Usare la lista tra graffe dopo la dichiarazione.

5

Confrontare due struct con == invece di confrontare i campi.

6

Confrontare due campi stringa con == invece di strcmp.

7

Calcolare la dimensione sommando i campi e dimenticare il padding.

8

Scrivere classe.media[i] invece di classe[i].media.

Riepilogo Concetti

Operazione	Tecnica	Note
Dichiarare il tipo	typedef struct { ... } Nome;	ogni struct è un tipo nuovo
Accedere a un campo	s.media	il campo si usa come una variabile
Inizializzare	{v1, v2}, {.campo = v}, {0}	solo nella dichiarazione
Copiare	b = a	anche gli array interni
Confrontare	campo per campo	strcmp per le stringhe
Dimensione	sizeof(Tipo), offsetof	il padding conta
Array di struct	classe[i].campo	un record per elemento
Ricerca e massimo	condizione su un campo	il risultato è un indice
Struct con array	classe[s].voti[v]	numero di elementi in un campo

SEZIONE 1

Esempi Pratici: Dichiarazione e Accesso

Concetti base — tre file su campi e inizializzazione.

1 – 010-tipo-con-campi.c

Problema

nome, classe e media di due studenti della 3B.

Tecnica

struct studente, operatore punto e sizeof.

2 – 020-inizializzazione.c

Problema

le tessere della mensa scolastica.

Tecnica

inizializzazione in ordine, con il nome dei campi e a zero.

3 – 030-typedef-struct.c

Problema

il costo del carburante per la gita.

Tecnica

Euro e Chilometri come struct: assegnamento che non compila.

SEZIONE 1

Esempi Pratici: Problemi su Dichiarazione e Accesso

040 – Scheda di un libro

Prestiti e restituzioni sulle copie disponibili della biblioteca.

050 – Distanza GPS

Distanza tra due Punto dell'uscita didattica e tempo a piedi.

060 – Durata delle lezioni

Due Orario per lezione: durata, anomalie e sovrapposizioni.

070 – Somma di frazioni

Dosi di una ricetta come Frazione, ridotte ai minimi termini.

080 – Carta d'imbarco

Controllo al gate di CartaImbarco e Volo, due tipi diversi.

SEZIONE 2

Esempi Pratici: Assegnamento e Confronto

Concetti base — tre file su copia, confronto e dimensione.

1 – 010-copia-con-assegnamento.c

Problema

simulazione dei voti di fine anno.

Tecnica

copia di una Pagella con un array interno.

2 – 020-confronto-campo-per-campo.c

Problema

inventario del laboratorio e conteggio di fine anno.

Tecnica

strcmp e == sui campi, == tra struct non compila.

3 – 030-dimensione-e-allineamento.c

Problema

letture dei sensori in una centralina con poca memoria.

Tecnica

padding, offsetof e ordine dei campi.

SEZIONE 2

Esempi Pratici: Problemi su Assegnamento e Confronto

040 – Scambio di posti

Due studenti si scambiano la postazione: scambio sbagliato e corretto.

050 – Backup delle impostazioni

LIM dell'aula: backup, ripristino e valori di fabbrica con un assegnamento.

060 – Confronto tra pagelle

Secondo quadrimestre come copia del primo, variazioni e recuperi.

070 – Modifiche all'anagrafica

Copia di lavoro, elenco dei campi cambiati e conferma.

080 – Doppioni nelle iscrizioni

Iscrizioni ai corsi pomeridiani confrontate con la precedente.

SEZIONE 3

Esempi Pratici: Array di Struct

Concetti base — tre file su tabelle di record.

1 – 010-array-di-struct.c

Problema

l'elenco della 3B con età e media.

Tecnica

Studente classe[5] e copia di un elemento.

2 – 020-ricerca-e-massimo.c

Problema

i libri della biblioteca di classe.

Tecnica

ricerca per titolo, massimo e conteggio su un campo.

3 – 030-struct-con-array.c

Problema

il registro dei voti di matematica.

Tecnica

classe[s].voti[v] e numero di voti in un campo.

SEZIONE 3

Esempi Pratici: Problemi su Array di Struct

040 – Rubrica di classe

Ricerca per inizio del cognome e per ruolo.

050 – Magazzino del laboratorio

Carico, prelievo e articoli sottoscorta da riordinare.

060 – Corsa campestre

Podio di ogni categoria con ricerche ripetute del minimo.

070 – Prenotazione delle aule

Una matrice di prenotazioni dentro ogni Aula.

080 – Registro delle assenze

Ore di assenza mensili e stima rispetto al limite annuale.